

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: B.Charishma

Roll No.: a24126552067

Date: 30-08-2026

1. TITLE

State Management and DOM Traversal using JavaScript Event Listeners

2. AIM

To build an interactive To-Do List application that demonstrates state management, class inheritance, dynamic element creation, DOM traversal, and event-driven architectural patterns.

3. OBJECTIVE

The objectives of this experiment are:

- To implement ES6 Class inheritance using `extends` and `super()`.
- To manage a global state counter for unique DOM element identification.
- To dynamically append HTML segments to a parent container.
- To pass specific identifying parameters into inline onclick event handlers.
- To remove and style specific DOM nodes upon user interaction.

4. THEORY

Interactive web applications rely heavily on event listeners and DOM manipulation. When a user interacts with the page (e.g., clicking a button), JavaScript captures the event and executes logic to modify the UI.

Class inheritance in JavaScript allows a child class to inherit properties and methods from a parent class, promoting code reusability. A child class calls `super()` to invoke the parent's constructor.

Dynamically generated elements must often carry unique identifiers so that subsequent interactions (like deleting or completing a specific item) know exactly which DOM node to target.

Application Flow:

User Input -> Create WebTask Instance -> Generate Unique HTML -> Append to DOM -> Handle Inline Events (Complete/Delete) -> Update State

5. ALGORITHM / PROCEDURE

1. Define a parent Task class with basic properties (id, text).
2. Define a child WebTask class that extends Task and adds a getHTML() method returning a formatted UI string.
3. Initialize a global taskCounter to track unique IDs.
4. Create an addTask() function that reads input, validates it, and instantiates a WebTask.
5. Append the generated HTML to the list container and increment the counter.
6. Create a completeTask(id) function that finds the specific text span by ID and applies inline CSS (line-through).
7. Create a deleteTask(id) function that finds the specific task box by ID and executes .remove().
8. Implement a helper function to toggle the visibility of an "Empty List" message based on container content.
9. Test adding, completing, and deleting multiple tasks.

6. PROGRAM / SOURCE CODE

Source File: index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>My To-Do List</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <h1>My To-Do List</h1>

  <input type="text" id="taskInput" placeholder="Enter a task">
  <button id="addButton">Add Task</button>

  <p id="emptyMessage">No tasks available.</p>

  <ul id="taskList"></ul>

  <script src="script.js"></script>
</body>
</html>
```

Source File: script.js

```
const taskInput = document.getElementById("taskInput");
const addButton = document.getElementById("addButton");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

addButton.addEventListener("click", function () {

    const taskText = taskInput.value.trim();

    if (taskText === "") {
        alert("Please enter a task.");
        return;
    }

    // Create new list item
    const li = document.createElement("li");

    // Create task text
    const span = document.createElement("span");
    span.textContent = taskText;

    // Create Complete button
    const completeButton = document.createElement("button");
    completeButton.textContent = "Complete";

    // Create Delete button
    const deleteButton = document.createElement("button");
    deleteButton.textContent = "Delete";

    // Complete button event
    completeButton.addEventListener("click", function () {
        span.classList.toggle("completed");
    });

    // Delete button event
    deleteButton.addEventListener("click", function () {
        li.remove();

        if (taskList.children.length === 0) {
            emptyMessage.style.display = "block";
        }
    });

    // Add elements to list item
    li.appendChild(span);
    li.appendChild(completeButton);
    li.appendChild(deleteButton);

    // Add list item to task list
    taskList.appendChild(li);
```

```

// Hide empty message
emptyMessage.style.display = "none";

// Clear input box
taskInput.value = "";
});

```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>class WebTask extends Task</code>	Creates a child class inheriting from the Task parent class.
<code>super(id, text);</code>	Invokes the parent class constructor to initialize inherited properties.
<code>document.getElementById('id').innerHTML += ...</code>	Appends a new HTML string to the end of an element's existing content.
<code>onclick='deleteTask(\${this.id})'</code>	Binds a dynamic integer parameter to an inline event handler.
<code>specificText.style.textDecoration = 'line-through';</code>	Applies an inline CSS style to visually strike out completed text.
<code>specificBox.remove();</code>	Completely detaches and deletes a node from the DOM.

8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Add New Task	Input text and click Add Task	New task UI block appended to DOM successfully	Pass
TC-02	Complete Task Action	Click Complete button on a specific task	Target task text becomes gray and struck-through	Pass
TC-03	Empty State Validation	Check state rendering	No tasks available message dynamically handled	Pass

9. OUTPUT

Program Main Output

My To-Do List

Learn MongoDB

Add Task

No tasks available.

10. TEST CASES SCREENSHOTS

Test Case Execution 1

My To-Do List

Invalid State Check

Add Task

No tasks available.

11. RESULT

Thus, the experiment was successfully implemented and verified across both positive functional and negative validation test cases.